

Um Arcabouço Para Provisionamento Automático de Recursos em Provedores de IaaS Independente do Tipo de Aplicação

Fábio Morais¹, Francisco Brasileiro¹, Raquel Lopes¹, Ricardo Araújo¹,
Augusto Macedo¹, Wade Satterfield², Leandro Rosa³

¹ Universidade Federal de Campina Grande
Laboratório de Sistemas Distribuídos, Campina Grande – Brasil

²Hewlett-Packard – ESSN – Fort Collins, Colorado Area – USA

³Hewlett-Packard – ESSN – Brazil Lab, Porto Alegre – Brasil

{fabio, fubica, raquel, ricardo, augustoq}@lsd.ufcg.edu.br
{Wade.Satterfield, Leandro.Rosa}@hp.com

Abstract. *Infrastructure cost reductions can be achieved by taking advantage of the elasticity provided by IaaS deployments. However, state-of-the-practice auto-scaling solutions use simple reactive approaches, which can successfully reduce these costs, but are often not efficient at minimizing SLA violations. In this paper we propose a flexible and non-intrusive framework for auto-scaling services that follows both a reactive and a proactive approach. Our framework uses a configurable set of predictors of future service demands and a selection mechanism that periodically decides the best predictor to be used. We also propose a prediction correction method that aims at minimizing underestimation errors, which may reduce the number of SLA violations. Simulation experiments using production traces from HP customers were carried out. Results show costs savings of as much as 37%, while the probability of an SLA violation can be kept, on average, as small as 0.008%, and no larger than 0.036%.*

Resumo. *Custos de infraestrutura podem ser reduzidos por meio da elasticidade oferecida pelos provedores de IaaS. No entanto, soluções do mercado utilizam abordagens reativas, que reduzem esses custos, mas não são suficientes para evitar quebras de SLA. Nesse trabalho, propomos um arcabouço para provisionamento automático de recursos, flexível e não intrusivo, que emprega abordagens reativa e proativa, baseadas no uso de um conjunto configurável de preditores de demandas dos serviços, além de um mecanismo de seleção que decide periodicamente o melhor preditor a ser usado. Também propomos uma nova maneira de corrigir previsões subestimadas, reduzindo por consequência o número de quebras de SLA. Nós avaliamos o arcabouço através de simulações que usam dados de produção de clientes da HP. Os resultados mostram que é possível obter uma economia de até 37% enquanto a probabilidade de quebra de SLA é mantida em média em 0,008% e limitada superiormente a 0,036%.*

1. Introdução

O modelo de infraestrutura como um serviço (IaaS, do inglês *infrastructure-as-a-service*) oferecido pelo mercado de *Computação na Nuvem* vem sendo, cada vez mais, usado pelo

departamento de TI (Tecnologia da Informação) das empresas para aquisição de capacidade computacional. Isso é feito atualmente de duas formas: (i) através de infraestruturas privadas, reduzindo o custo da aquisição por meio da economia de escala de infraestruturas próprias; ou (ii) com o uso de infraestruturas públicas, nas quais a capacidade é adquirida sob demanda de provedores externos de IaaS, que normalmente empregam a tarifação do serviço baseada no modelo “pague conforme utilização” (do inglês *pay-as-you-go*), no qual clientes pagam apenas pelos recursos de fato utilizados.

Uma das principais vantagens oferecidas pelos provedores de IaaS para os clientes é a elasticidade¹ no provisionamento de capacidade a curto prazo. Uma estratégia ótima para executar serviços com demandas que variam no tempo através de recursos virtuais deve prover, em qualquer instante de tempo, a capacidade exata para manter o desempenho do serviços e satisfazer os acordos de nível de serviço (SLA, do inglês *service level agreement*). Tal abordagem gerencia automaticamente a capacidade, provendo mais recursos quando a demanda aumenta, e liberando recursos à medida que não são mais necessários.

Um aspecto importante de um mecanismo de provisionamento automático eficiente é a habilidade de antecipar mudanças na demanda do serviço. Monitoramento e predição são tarefas importantes que precisam ser cuidadosamente realizadas pelo mecanismo. No entanto, estimar a demanda é uma tarefa não trivial que tem recebido pouca atenção nas soluções de planejamento de capacidade a curto prazo, tando do mercado como da academia. Em geral, essas soluções atuam reagindo a mudanças na carga [Amazon 2012, RightScale 2012, Marshall et al. 2010, Vijayakumar et al. 2010, Calcavecchia et al. 2012], ou assumem a disponibilidade de um preditor sem de fato avaliar a influência de tais preditores na performance do mecanismo de provisionamento automático em si.

Além do mais, alguns trabalhos requerem a coleta de informações sobre o serviço em execução na infraestrutura (como tempo de resposta, tamanho da fila, taxa de chegada e demanda das requisições) [Urgaonkar et al. 2008, Sharma et al. 2011, Vijayakumar et al. 2010]. A dependência dessas informações pode inviabilizar esse tipo de solução de provisionamento, visto que, em alguns casos, essas são informações críticas que nem sempre podem ser compartilhadas, mesmo com os provedores de IaaS.

Neste artigo nós abordamos limitações existentes em trabalhos anteriores propondo um mecanismo proativo e reativo de provisionamento automático que: é *independente do tipo de aplicação* (do inglês *service-agnostic*) — necessita monitorar apenas os recursos da infraestrutura onde a aplicação está em execução, ou seja, o mecanismo é não intrusivo; e possui *configuração dinâmica e automática* em relação às atividades de predição e de monitoramento. Em particular, o mecanismo considera o uso de múltiplos preditores, dado que não existe um único preditor que é o melhor em estimar a carga para todas as aplicações [Jiang et al. 2012]. Um seletor avalia continuamente as opções de predição que estão disponíveis e criteriosamente escolhe o preditor para ser utilizado no futuro próximo, permitindo que o mecanismo se adapte a alterações nos padrões de carga. Além disso, propomos um novo método de correção de predição que visa evitar erros de subprovisionamento e diminuir o número de quebras de SLA.

¹Propriedade que permite alterar dinamicamente a capacidade da infraestrutura.

2. Trabalhos Relacionados

Gerência de recursos em infraestruturas virtualizadas é um tema abordado por diversas pesquisas, que em geral levam em consideração uma ou mais das seguintes atividades:

- Planejamento de capacidade a longo prazo: decide quantos recursos são necessários para compor a infraestrutura no longo prazo (pelo menos um ano à frente) [Maciel-Jr. et al. 2011];
- Planejamento de capacidade a médio prazo: decide quantos recursos físicos devem ser ligados nas próximas horas ou dias, com o objetivo principal de reduzir custos de energia [Jiang et al. 2012];
- Planejamento de capacidade a curto prazo: decide quantas máquinas virtuais (VM, do inglês *virtual machine*) são necessárias para implantar uma dada aplicação, ou serviço, nos próximos minutos [Calcavecchia et al. 2012, Urgaonkar et al. 2008, Marshall et al. 2010, Sharma et al. 2011];
- Escalonamento de VM: decide onde criar as VMs que são necessárias, dentre os recursos físicos disponíveis, tentando maximizar a utilização de recursos e reduzir os custos, sem comprometer o desempenho [Shen et al. 2011, Almeida et al. 2010, Han et al. 2012].

Todas estas áreas estão fortemente relacionadas. A gerência de capacidade a curto prazo precisa instanciar VMs sobre os recursos físicos disponíveis, o que é realizado pelo gerente de escalonamento de VM. O gerente de escalonamento de VM utiliza os recursos físicos ligados de acordo com a decisão tomada pelo gerente de capacidade a médio prazo que, por sua vez, controla os recursos físicos disponíveis decorrentes da decisão tomada pelo gerente de capacidade a longo prazo. Este trabalho concentra-se no planejamento a curto prazo.

Ao analisar em profundidade soluções de planejamento de capacidade a curto prazo, podemos compará-las segundo dois aspectos: o modo de operação e o nível de intrusão. Alguns gerentes são proativos em seu modo de operação [Sharma et al. 2011], tentando estimar cargas futuras e aumentando ou diminuindo a capacidade dos serviços quando necessário. Outros, apenas reagem a mudanças na demanda [Marshall et al. 2010, Calcavecchia et al. 2012, Merino et al. 2010], e outros seguem uma abordagem híbrida [Urgaonkar et al. 2008], proativa e reativa, realizando previsões de carga e reagindo, o mais rápido possível, às más decisões de planejamento de capacidade. O mecanismo proposto nesse artigo segue esta última abordagem.

Quanto ao aspecto da intrusão, alguns gerentes a curto prazo são dependentes de aplicação [Urgaonkar et al. 2008, Seung et al. 2011, Marshall et al. 2010] no sentido de que eles precisam de informações específicas sobre os serviços que estão sendo providos, como tempos de resposta, tamanhos de filas, taxas de chegada e demandas de requisições. Esta exigência não só levanta questões de privacidade, como também traz mais complexidade à estrutura de gestão, uma vez que o serviço deve ser instrumentado para ser monitorado corretamente.

Por outro lado, existem gerentes que requerem conhecimento sobre o que está sendo provisionado. Por exemplo, gerentes a curto prazo que necessitam apenas do monitoramento da carga das VMs, mas assumem que os serviços seguem a estrutura característica de processamento em lote (do inglês *batch*) [Calcavecchia et al. 2012]. Nós

propomos um mecanismo que é independente do tipo de aplicação e que não exige informações específicas sobre a mesma, nem possui o requisito de que o serviço seja de um determinado tipo. Nosso mecanismo necessita apenas do histórico de utilização da infraestrutura virtual, o que pode ser facilmente obtido no nível da VM.

Até onde sabemos, quando considerado o provisionamento horizontal, o mecanismo de provisionamento automático que propomos nesse artigo é o primeiro a ser totalmente independente do serviço que está sendo provido. Além disso, ele segue um modo de operação proativo e reativo, onde o comportamento proativo considera a utilização de múltiplos preditores, criteriosamente selecionados de tempos em tempos para prover estimativas das demandas futuras do serviço, permitindo ao provisionamento automático a capacidade de adaptação a mudanças no padrão das cargas.

3. Definição do Problema

Consideramos que o centro de processamento de dados que dá suporte ao modelo de IaaS é composto por um número de servidores que utilizam alguma tecnologia de virtualização. Esses servidores abrigam as VMs que executam os serviços dos clientes do provedor de IaaS. O centro de processamento de dados define um conjunto de tipos de instâncias que podem ser providas por esses servidores. Cada tipo de instância caracteriza uma VM em termos de capacidade de recursos (CPU, memória RAM, largura de banda, etc.).

Os serviços, ou aplicações, que executam na infraestrutura são compostos por uma ou mais camadas. Cada uma destas camadas pode apresentar cargas de trabalho que variam no tempo, o que significa que a quantidade de VMs necessárias para executar adequadamente uma camada do serviço pode variar com o tempo. Desta forma, cada camada do serviço é vista como um componente que deve ser provisionado independentemente. Assumimos que cargas de trabalho são escaláveis horizontalmente e que um serviço de balanceamento de carga é capaz de balancear adequadamente a demanda da camada em todas as VMs que foram alocadas para essa camada. Por uma questão de simplicidade, assume-se que cada camada do serviço está associada a um tipo de instância, ou seja, uma camada do serviço só pode ser executada num único tipo de instância, enquanto camadas diferentes podem ser executadas por tipos diferentes.

Cada camada do serviço está associada a um ou mais objetivos de nível de serviço (SLO, do inglês *service level objective*). Assumimos que existe um mapeamento que relaciona a satisfação dos SLOs com a utilização de recursos das VMs alocadas ao serviço, de tal forma que se a utilização do recurso é mantida abaixo de um valor alvo na maior parte do tempo, então os SLOs da camada do serviço são satisfeitos. É certo que, quanto mais próxima do alvo a utilização de recursos for mantida, menor é o número de máquinas virtuais necessárias e menor é o custo para executar esta camada do serviço com o nível de desempenho desejado.

Portanto, o objetivo do mecanismo de provisionamento automático é realizar o planejamento de capacidade a curto prazo para cada camada do serviço em execução no centro de processamento de dados, com o intuito de minimizar a quantidade de recursos ativos, além de satisfazer os SLOs das camadas.

4. Um Arcabouço Para Provisionamento Automático de Recursos

4.1. Arquitetura

A nossa solução baseia-se num laço de controle com retroalimentação (do inglês *feedback control loop*). A Figura 1 mostra a arquitetura conceitual. Adicionamos ao sistema um monitor responsável por coletar informações sobre a utilização dos recursos no nível de VM. Ou seja, coletar informações sobre a utilização de cada VM ativa. Assumimos que o monitor é capaz de monitorar um grupo restrito de recursos como consumo de CPU, memória, largura de banda, etc. Informações sobre diferentes recursos podem ser coletadas para cada camada do serviço com periodicidade configurável. A informação coletada é periodicamente comparada a valores de referência (por exemplo, nível desejado de utilização) definidos pelo administrador da infraestrutura e pelo cliente. As diferenças entre os valores coletados e os de referência, ou seja, os erros medidos, são repassados ao controlador, que atua no sistema para trazê-lo a um estado desejado, no qual tais erros se aproximem de zero. No nosso caso, o controlador atua no sistema iniciando novas VMs ou parando VMs em execução para manter a utilização dos recursos o mais próximo possível dos valores alvo associados a esses recursos.

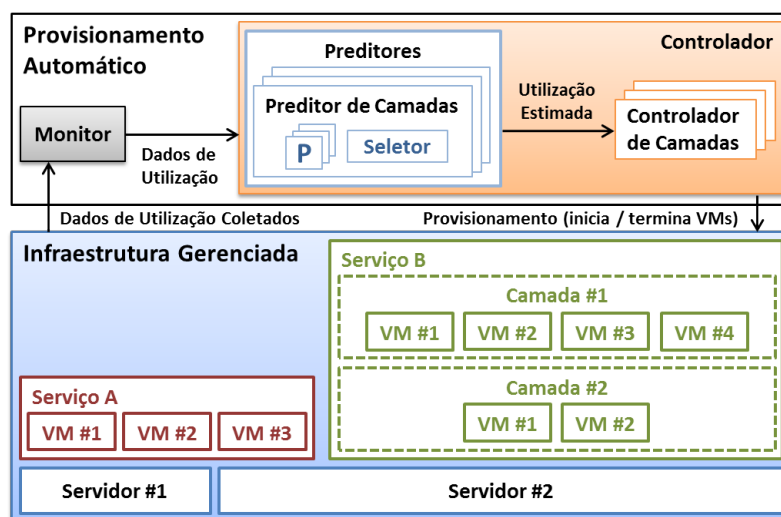


Figura 1. Arquitetura do Arcabouço para Provisionamento Automático

O controlador é composto por controladores de camada que executam periodicamente, em intervalos configuráveis, para realizar planejamento de capacidade a curto prazo de cada camada. Os controladores de camada operam de forma reativa e proativa. O comportamento reativo é implementado definindo ações associadas a cada tipo de recursos usado numa determinada camada. Essas ações são executadas toda vez que a utilização dos recursos atingir um dado limite. O comportamento proativo, por sua vez, é implementado da seguinte forma: para cada camada do serviço e tipo de recurso usado para controlar uma camada em particular, existe um conjunto de preditores para estimar a utilização futura do recurso para a camada. Esses preditores consomem a informação coletada sobre a utilização dos recursos e, baseados nessa informação, estimam a demanda futura da camada para o recurso específico. Esses preditores diferem apenas na estratégia de predição usada, ou seja, como o preditor usa a informação coletada para estimar a carga futura. Preditores se registram no monitor do sistema para receber as informações

específicas que necessitam. Controladores de camadas diferentes podem usar diferentes conjuntos de preditores.

Em cada instante de tempo e para cada tipo de recurso usado para controlar a capacidade de uma camada, apenas um dos preditores independentes é usado para estimar a utilização futura de um dado recurso. De tempos em tempos, um módulo seletor tenta descobrir qual o preditor mais apropriado para usar num futuro próximo, dentre o conjunto de preditores disponíveis para uma camada particular. Essa seleção é importante pois, como mostraremos mais à frente, não há um único preditor que seja eficiente para estimar a ampla variedade de padrões de utilização existentes na prática, tanto ao comparar camadas distintas, como para uma única camada, especialmente ao considerar janelas de tempo mais longas.

Cada controlador de camada calcula o número de VMs necessárias no próximo intervalo de controle para manter a camada em execução, baseado nos valores de referência de utilização e nas estimativas para futuros valores de demanda dos recursos usados pela camada. Se a quantidade necessária de VMs é maior do que a atualmente alocada para a execução da camada, VMs adicionais são iniciadas. Caso contrário, a ação a ser tomada depende do tipo de infraestrutura provida. Em um modelo público de IaaS, por exemplo, os provedores cobram geralmente um preço fixo por VM usada numa unidade de tempo pré-definida, por exemplo US\$ 0.10 por hora. Nesse caso, o controlador deve somente desligar VMs que estejam próximas de completar a hora. Por outro lado, num modelo privado de IaaS, VMs devem ser desligadas o mais cedo possível. Obviamente, deve haver algum tipo de orquestração entre os controladores de camadas, tal que decisões tomadas por um controlador não afetem outras camadas do mesmo serviço e que pedidos simultâneos de criação de VMs sejam devidamente satisfeitos pela infraestrutura física. Evidentemente, modelos de orquestração mais sofisticados podem ser implementados de modo a melhorar a eficiência do sistema. Isso, no entanto, está fora do escopo desse trabalho.

Diferente de outras abordagens, nossa solução realiza planejamento de capacidade baseado somente em informações históricas da infraestrutura usada pelas camadas do serviço e não requer nenhuma informação sobre o serviço em si, além do mapeamento entre as configurações particulares de VMs e as camadas do serviço. Nesse sentido, a solução é independente do serviço em execução na infraestrutura, além de prover mais flexibilidade na forma com a qual as predições são feitas, dado que é possível incorporar quantos preditores se queira, por camada, sem a necessidade de alterar o funcionamento do seletor.

4.2. Escolhendo Dinamicamente Entre Múltiplos Preditores

4.2.1. Modelo de Simulação

A decisão de usar múltiplos preditores selecionados dinamicamente ao longo do tempo baseia-se em evidências empíricas de que é melhor do que usar um único preditor, mesmo que este preditor combine características de diversos outros, como quando é utilizada uma abordagem de agregação [Jiang et al. 2012]. O impacto dessa decisão não inviabiliza o uso do arcabouço, visto que experimentos usando a linguagem R [Chambers 2012] mostraram que em 50% dos casos o tempo de predição é inferior a 0,15 segundos e em 99% dos casos não é maior que 13,5 segundos.

Implementamos um modelo de simulação do arcabouço apresentado na subseção anterior também usando a linguagem R ². O simulador é alimentado com arquivos de dados de utilização de aplicações, em produção, de clientes da HP. Consideramos que cada arquivo de dados corresponde a uma camada diferente do serviço. Visto que cada controlador de camada é independente e que o planejamento de capacidade do provedor IaaS não é nosso foco, cada controlador de camada é simulado separadamente.

Os arquivos de dados provêm itens coletados a cada 5 minutos. Cada item é um par (u, c) , onde u é a utilização média de CPU no intervalo de 5 minutos e c é o número de núcleos de CPU usados. Um novo par lido é usado para computar a utilização real das VMs no sistema simulado para os próximos 5 minutos. Consideramos VMs com um único núcleo de CPU. Seja t o instante de tempo no qual um novo par (u, c) é lido e a o número de VMs alocadas para executar a camada do serviço no experimento de simulação, a utilização real de cada VM do instante de tempo t até $t + 5$ minutos é computada como o mínimo entre 100% e $u \cdot c/a$. Além disso, se $u \cdot c/a > 1$, então o excesso da demanda que não pôde ser alocado no tempo t (dado por $1 - u \cdot c/a$) é adicionado à demanda do próximo intervalo de tempo, que ocorre em $t + 5$ minutos.

A decisão de adicionar ou remover VMs pode ser tomada proativamente a cada 5 minutos ou reativamente toda vez que uma determinada condição acontecer. Por exemplo, a utilização de CPU está acima de um dado limite. No entanto, dado que é necessário um tempo até que o provisionamento das novas VMs tenha efeito, no modelo de simulação consideramos que se um controlador de camada decidir no tempo t que uma nova VM deve ser alocada, tal VM estará de fato operacional apenas no tempo $t + 5$. Dessa forma, no caso proativo, os preditores no controlador de camada usam informações históricas coletadas até o tempo t para estimar a demanda no tempo $t + 5$ minutos e usam essa estimativa para decidir, no tempo t , quantas VMs devem estar executando no tempo $t + 5$ minutos. Portanto, qualquer quantidade extra de VMs necessária no tempo $t + 5$ minutos é requisitada no tempo t . Foram utilizados intervalos de 5 minutos em conformidade com os dados de utilização usados para avaliar o arcabouço. No entanto, esse intervalo é configurável e, idealmente, pode ser um valor representativo do tempo necessário para tornar uma nova VM operacional.

Nos experimentos de simulação reportados nesse artigo, consideramos apenas o modelo público de IaaS e um modelo de tarifação que cobra por hora pelo uso de VMs. Ou seja, VMs são terminadas somente em horas completas, como discutido na Subseção 4.1.

4.2.2. Camadas diferentes necessitam de diferentes preditores

Para esse estudo inicial, cada controlador de camada tem um único preditor, o que torna o mecanismo de seleção trivial. O controlador trabalha para manter a utilização de CPU abaixo do limite de 70%. Avaliamos 5 preditores populares, baseados em: autocorrelação (AC), regressão linear (LR), autorregressão (AR), autorregressão com média móvel integrada (ARIMA) e no valor da medição anterior (LW, do inglês *last window*). Também avaliamos um sexto preditor que usa os outros 5 preditores em conjunto (EN), como proposto por Jiang et al. [Jiang et al. 2012].

²O código-fonte encontra-se disponível no endereço <http://www.lsd.ufcg.edu.br/~fabio/autoflex>.

Usamos dois conjuntos de dados de utilização de produção, um com 265 arquivos com um mês de duração e outro com 33 arquivos com média de 8 meses de duração. A função distribuição acumulada das utilizações de CPU desses dados é respectivamente mostrada nas Figuras 2(a) e 2(b). Nota-se que os dados apresentam uma ampla variedade de distribuições de utilização em ambos os casos. Isso também acontece para a função distribuição acumulada (FDA) para as variações de utilização de CPU, definidas como a diferença entre as utilizações no tempo t e no tempo $t + 5$ minutos. Dessa forma, consideramos que os dados utilizados no estudo são representativos.

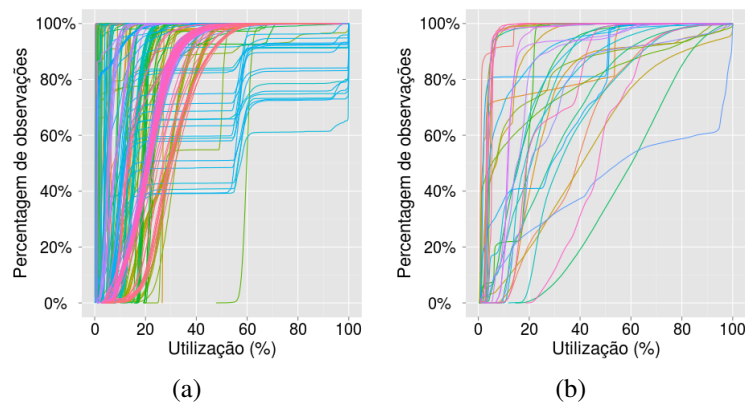


Figura 2. FDA da utilização de CPU para os dois conjuntos de dados de utilização, com duração de um mês (a) e com duração média de 8 meses (b).

A partir da simulação de cada um dos 265 arquivos, com um mês de duração, medimos o número de intervalos de 5 minutos em que a utilização de CPU foi de 100% e havia demanda que não pôde ser atendida, e foi, portanto, deslocada para o próximo intervalo. Isso representa o número de intervalos de tempo onde a capacidade provida não foi suficiente para suprir a demanda do serviço³. Chamamos esses eventos de *violações* de SLO do serviço. Medimos também a economia conseguida quando comparamos o provisionamento automático com uma abordagem que realiza provisionamento perfeito, ou seja, uma abordagem que conhece a demanda mais alta no período simulado e estaticamente aloca recursos tal que a utilização seja sempre menor que um dado limite (70% em nossos experimentos). Essas métricas foram utilizadas independentemente para comparar o desempenho dos vários preditores usados. Na Figura 3 apresentamos, para cada preditor, a percentagem dos dados de utilização para os quais um dado preditor foi avaliado como o melhor⁴, considerando separadamente ambas as métricas.

Como pode ser visto, quando consideramos o número de violações de SLO, o preditor com melhor desempenho responde apenas por 44% dos casos, enquanto esse número aumenta para 64% quando usamos a métrica de custo para ordenar os preditores. Dessa forma, é possível perceber que utilizar múltiplos preditores tem um impacto direto no desempenho alcançado com o mecanismo.

³Ressaltamos que só foi possível computar essa métrica por se tratar de um experimento simulado. Em um ambiente real, não há como medir a demanda real quando a utilização está em 100%.

⁴Lembramos que a soma das partes no gráfico pode ser maior que 100% dado que, para alguns dos dados, vários preditores foram igualmente bons.

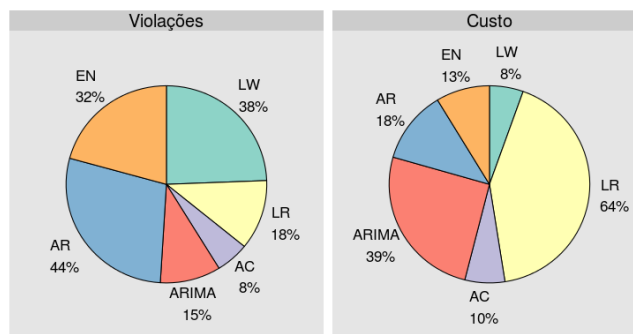


Figura 3. Percentagem de vezes em que um dado preditor é avaliado como o melhor, considerando os 265 diferentes arquivos de dados de utilização

4.2.3. Uma Camada Requer Diferentes Preditores ao Longo do Tempo

Avaliamos também como a qualidade de um preditor varia ao longo do tempo ao predizer demandas de um mesmo serviço. Para tal, alimentamos as simulações com os 33 arquivos de dados de produção que, em média, possuem aproximadamente 8 meses de duração.

Nesse caso, o conjunto de preditores incluem todos os seis preditores usados no experimento anterior. Um novo preditor é selecionado no início de todo período de 24 horas. Um seletor perfeito é usado para escolher o preditor mais apropriado. Isso foi possível alimentando os preditores com dados de utilização das duas semanas anteriores e usando a utilização do próximo dia para medir o desempenho de cada preditor⁵. O seletor então escolhe o melhor preditor de acordo com a métrica definida. As Figuras 4(a) e 4(b) apresentam, respectivamente, as funções distribuição acumulada do número de diferentes preditores escolhidos por arquivo de dados e do número de vezes que o preditor a ser usado é trocado pelo seletor ao longo do tempo, para os 33 arquivos simulados.

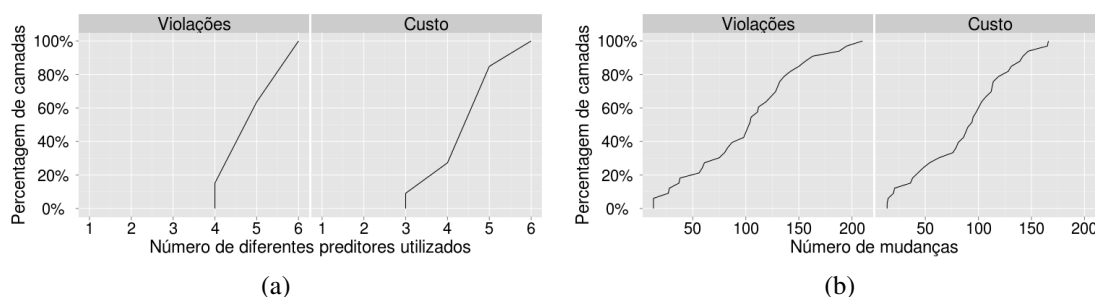


Figura 4. Número de diferentes preditores escolhidos pelo seletor (a) e a quantidade de vezes que o seletor troca de preditor (b).

Podemos ver que no mínimo 3 preditores diferentes são sempre utilizados e que o número de vezes que o seletor opta por trocar o preditor utilizado varia de dezenas a centenas de vezes durante o período de 8 meses. Isso indica que o desempenho do mecanismo pode ser melhorado se os preditores forem dinamicamente selecionados para uso à medida que o tempo avança e o padrão de utilização muda.

⁵Note que esse seletor não é implementável dado que ele requer conhecimento perfeito sobre utilização futura; ainda assim, ele é útil para o propósito de avaliação que realizamos nesse experimento.

5. Instanciação e Avaliação do Mecanismo de Provisionamento Automático

Nessa seção avaliamos uma implementação do arcabouço de uso geral proposto na seção anterior.

A fim de definir com precisão como o controlador irá operar, a instanciação do arcabouço requer as seguintes ações: (i) definir os limites e as ações associadas ao comportamento reativo do controlador; (ii) prover implementações para o conjunto de preditores a serem utilizados por cada controlador de camada; (iii) prover a implementação de um mecanismo de seleção para dinamicamente escolher o preditor a ser usado em cada instante de tempo; (iv) definir os valores apropriados para os parâmetros do arcabouço, tais como a periodicidade de execução dos controladores de camada e dos seletores e a frequência com que as informações monitoradas são coletadas.

A seguir, propomos uma maneira de implementar o seletor, preditores que tentam reduzir os erros que conduzem ao subprovisionamento e ações reativas simples. Desta forma, instanciamos o arcabouço de provisionamento automático visando reduzir substancialmente o número de violações, mas sem comprometer demasiadamente a economia de recursos que pode ser alcançada através da abordagem de provisionamento automático.

5.1. Como Selecionar o Preditor Apropriado?

Nossa implementação do módulo de seleção de preditor usa dados históricos de utilização para avaliar qual dentre os preditores disponíveis teria o melhor desempenho, com base no passado recente. Para isso, nós utilizamos as últimas três semanas de histórico de utilização. As duas semanas mais antigas são utilizadas para alimentar os preditores, enquanto que a semana mais recente de dados é utilizada como base para avaliar o desempenho dos preditores. O preditor que proporciona o melhor desempenho em termos do número de violações (e de custos, para o caso de empates) é selecionado para ser utilizado até que uma nova seleção seja realizada. Nos experimentos reportados a seguir uma nova seleção é realizada a cada dia. Outros valores de periodicidade de seleção foram experimentados, no entanto, usando o teste estatístico de Kruskal-Wallis [Kruskal and Wallis 1952], não foram constatadas diferenças significativas nos resultados obtidos para o número de violações de SLO.

Para avaliar o desempenho deste seletor, executamos simulações usando as mesmas configurações descritas da Subseção 4.2.3. Além do seletor apresentado anteriormente, consideramos para comparação dois outros seletores: o seletor perfeito, não realista, descrito na Subseção 4.2.3 e um seletor randômico, que aleatoriamente seleciona o preditor para ser usado. Estes podem ser vistos como limites superior e inferior, respectivamente, sobre o desempenho que pode ser obtido pelo nosso seletor. Na Figura 5 apresentamos o diagrama de caixa do número de violações ao simular as 33 cargas de trabalho de longa duração. Como pode-se ver, o método proposto é ligeiramente pior que o perfeito e muito melhor que a seleção randômica. Para todos os casos, a redução de custos quando comparada com um sistema superprovisionado é em torno de 65%.

5.2. Melhorando Predições Para Evitar Subestimativas

Verificamos que mesmo se o seletor perfeito for utilizado com o conjunto padrão de preditores considerado nesse artigo, o número de violações ainda pode ser muito elevado.

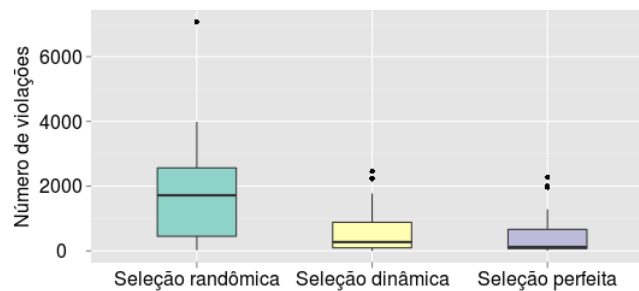


Figura 5. Avaliação do método de seleção dinâmica, de acordo com o número de violações, em comparação com a seleção randômica e a seleção perfeita.

Violações de SLO podem levar a quebras de SLA, que em geral geram custos elevados. Assim, em alguns casos, é importante reduzir, tanto quanto possível, o número de tais eventos, mesmo que isso implique em um aumento no custo final de provisionamento. Uma maneira de alcançar esse objetivo no provisionamento automático é através da utilização de preditores mais conservadores. Essencialmente, esses são preditores que tentam corrigir suas previsões com base nos erros cometidos anteriormente. Em particular, considerando-se os erros que conduzem à subestimativa da capacidade necessária e consequentemente à ocorrência de violações [Gong et al. 2010].

Aqui propõe-se uma abordagem para corrigir previsões, de modo a reduzir o número de subestimativas. O processo de correção calcula o histórico de erros de previsão e tenta correlacionar a sequência recente de erros de previsão com sequências de erros de previsões no passado, através de uma função de autocorrelação (ACF, do inglês *autocorrelation function*). O ponto do passado em que a correlação é máxima é utilizado como o ponto central de uma janela de valores de erro. Então, o maior erro negativo dentro desta janela é utilizado para corrigir a previsão seguinte. Tanto o tamanho da sequência, como o tamanho da janela são parâmetros do mecanismo. Utilizamos um tamanho de sequência de 2 semanas e valores para o tamanho da janela que variam de 4 horas a 2 dias.

A Figura 6(a) mostra o diagrama de caixa do número de violações em simulações alimentadas com as cargas de trabalho de longa duração, utilizando uma configuração semelhante ao experimento anterior, mas incluindo, além dos seis preditores originais, versões de todos os preditores com o método de correção proposto acima e utilizando diferentes tamanhos de janela (referenciados na figura como “ACF- x ”, onde x é o tamanho da janela em intervalos de 5 minutos). Mostramos também o desempenho alcançado comparado ao método de correção proposto por Gong et al. [Gong et al. 2010] (referenciado como “padding”). Embora este método tenha sido proposto em um contexto diferente, ele é utilizado aqui para comparação, pois não encontramos outro método usado no mesmo contexto do nosso.

Pode-se observar que todos os métodos de correção são capazes de reduzir substancialmente o número de violações, sendo o método “ACF- x ” mais eficiente para a configuração que nós consideramos. Conforme esperado, o número de violações do método “ACF- x ” melhora quando o valor de x aumenta. A Figura 6(b) mostra a redução do custo correspondente para o mesmo experimento. É possível observar que a redução no número de violações ocorre às custas de um aumento no custo final de provisionamento.

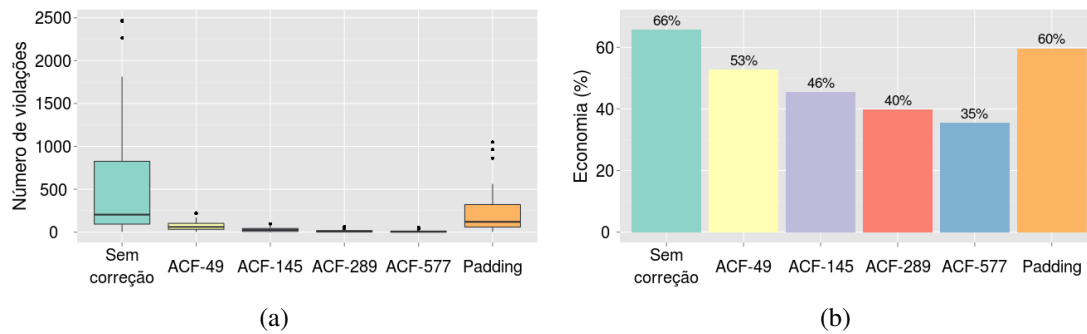


Figura 6. Comparação dos dois métodos de correção de predição com relação ao número de violações (a) e à economia de recursos (b).

5.3. Agrupando Todos os Mecanismos

É possível ver a correção de predições como um novo tipo de preditor. Ao fazer isso, consideramos as correções no arcabouço proposto, simplesmente incluindo no conjunto de preditores disponíveis alguns preditores que implementam procedimentos de correção. Logo, resultados diferentes podem ser obtidos quando se é mais ou menos conservador na configuração da instância do arcabouço de provisionamento automático.

Em seguida analisamos a relação de compromisso entre a redução do número de violações e a redução do custo de provisionamento envolvida na configuração de uma instância do arcabouço proposto. Executamos experimentos de simulação que realizam provisionamento automático proativo utilizando o seletor apresentado na Seção 5.1 e consideramos diferentes conjuntos de preditores. Seja P qualquer um dos preditores no conjunto $\{AC, LR, AR, ARIMA, LW, EN\}$, e $P-x$ um preditor que utiliza o preditor P e realiza correções através do método “ACF- x ” com uma janela de tamanho x , utilizamos os seguintes conjuntos de preditores: $C_1=\{P, P-49\}$, $C_2=\{P, P-49, P-145\}$, $C_3=\{P, P-49, P-145, P-289\}$, $C_4=\{P, P-49, P-145, P-289, P-577\}$, $C_5=\{P-49, P-145, P-289, P-577\}$, $C_6=\{P-145, P-289, P-577\}$, $C_7=\{P-289, P-577\}$, $C_8=\{P-577\}$. Em todos os casos, o provisionamento automático reativo é disparado sempre que uma violação ocorre. Neste caso, o preditor que realiza as maiores correções é utilizado até que o período corrente de 24 horas termine e uma nova seleção seja realizada pelo mecanismo de provisionamento automático proativo.

A Figura 7 apresenta o desempenho de diferentes configurações com relação ao número de violações e à redução de custo. Nós também apresentamos o desempenho da configuração que não realiza correção de erros de predição, para servir como caso base para comparação.

Como pode ser visto, as configurações que contêm $P-577$ são aquelas que proporcionam as maiores reduções no número de violações. Em média, quando comparado com o caso base, elas produzem um número de violações que é entre 31 e 122 vezes menor. Isto vem com uma redução na economia média dos custos alcançados que varia de 29% a 43%, quando comparado com a redução na economia média dos custos obtidos no cenário do caso base, mas que ainda rende substanciais economias de custo, variando de 37% a 46%. Por outro lado, as outras configurações apresentam menores reduções do número de violações, variando de 10 a 27 vezes, com reduções na economia média dos custos

alcançados de não mais que 28%.

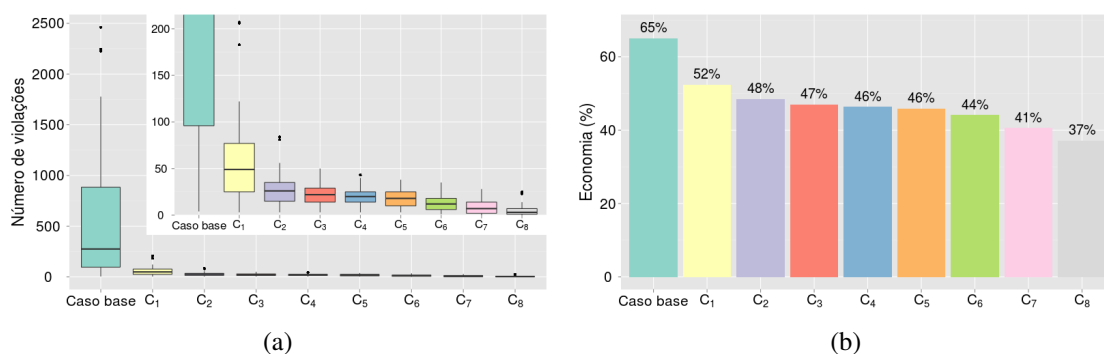


Figura 7. Comparação de diferentes configurações com relação ao número de violações (a) e à redução de custos (b).

6. Conclusão e Trabalhos Futuros

Propusemos neste artigo um novo arcabouço para a implementação de mecanismos de provisionamento automático que seguem abordagens tanto reativas quanto proativas, e que é independente do tipo de aplicação. A flexibilidade e a eficiência do arcabouço foram demonstradas por meio de experimentos de simulação conduzidos com o uso de dados de utilização de aplicações de clientes da HP. Os resultados destes experimentos mostram que uma configuração adequada do arcabouço proposto é capaz de reduzir a probabilidade média de ocorrência de violações para algo um pouco inferior a 0,008%, e gerando uma economia média de custos de 37%, quando comparado a um sistema super-provisionado. Além disto, para esse caso a probabilidade de ocorrer violações é sempre mantida abaixo de 0,036%. Economias mais substanciais podem ser alcançadas com um pequeno aumento na probabilidade de ocorrência de violações.

Como trabalho futuro, estamos avaliando como o desempenho do mecanismo de provisionamento automático pode ser melhorado através da combinação de dados de utilização de vários recursos diferentes. Além disso, estamos implementando o arcabouço proposto para avaliar seu desempenho em um ambiente real.

Agradecimentos

Essa pesquisa foi realizada em cooperação com Hewlett-Packard Brasil Ltda usando incentivos da Lei de Informática (Lei N° 8.248 de 1991). Francisco Brasileiro é pesquisador do CNPq (processo 305858/2010-6).

Referências

- Almeida, J., Almeida, V., Ardagna, D., Cunha, I., Francalanci, C., and Trubian, M. (2010). Joint admission control and resource allocation in virtualized servers. *J. Parallel Distrib. Comput.*, 70(4):344–362.
- Amazon (2012). Amazon auto scaling. <http://aws.amazon.com/autoscaling/>. Online; novembro, 2012.
- Calcavecchia, N., Capraescu, B., Di Nitto, E., Dubois, D., and Petcu, D. (2012). DEPAS: a decentralized probabilistic algorithm for auto-scaling. *Computing*, 94:701–730.

- Chambers, J. (2012). The R project for statistical computing. <http://www.r-project.org/>. Accessed: 19/11/2012.
- Gong, Z., Gu, X., and Wilkes, J. (2010). PRESS: PRedictive Elastic ReSource Scaling for cloud systems. In *Network and Service Management (CNSM), 2010 International Conference on*, pages 9–16. IEEE.
- Han, R., Guo, L., Ghanem, M. M., and Guo, Y. (2012). Lightweight resource scaling for cloud applications. In *Proceedings of the 2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (ccgrid 2012)*, pages 644–651, Washington, DC, USA.
- Jiang, Y., Perng, C.-s., Li, T., and Chang, R. (2012). Self-adaptive cloud capacity planning. In *Proceedings of the 2012 IEEE Ninth International Conference on Services Computing*, pages 73–80, Washington, DC, USA.
- Kruskal, W. and Wallis, W. (1952). Use of ranks in one-criterion variance analysis. *Journal of the American statistical Association*, 47(260):583–621.
- Maciel-Jr., P. D., Brasileiro, F. V., Lopes, R. V., Carvalho, M., and Mowbray, M. (2011). Evaluating the impact of planning long-term contracts on the management of a hybrid IT infrastructure. In *Integrated Network Management*, pages 89–96.
- Marshall, P., Keahey, K., and Freeman, T. (2010). Elastic site: Using clouds to elastically extend site resources. In *Proceedings of the 2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*, pages 43–52, Washington, DC, USA.
- Merino, L. R., Vaquero, L. M., Gil, V., Galán, F., Fontán, J., Montero, R. S., and Llorente, I. M. (2010). From infrastructure delivery to service management in clouds. *Future Generation Computer Systems*, 26(8):1226 – 1240.
- RightScale (2012). Right scale cloud management. <http://www.rightscale.com>. Online; novembro, 2012.
- Seung, Y., Lam, T., Li, L. E., and Woo, T. (2011). Cloudflex: Seamless scaling of enterprise applications into the cloud. In *INFOCOM*, pages 211–215.
- Sharma, U., Shenoy, P., Sahu, S., and Shaikh, A. (2011). A cost-aware elasticity provisioning system for the cloud. In *Proceedings of the 2011 31st International Conference on Distributed Computing Systems*, pages 559–570, Washington, DC, USA.
- Shen, Z., Subbiah, S., Gu, X., and Wilkes, J. (2011). CloudScale: elastic resource scaling for multi-tenant cloud systems. In *Proceedings of the 2nd ACM Symposium on Cloud Computing*, pages 5:1–5:14, New York, NY, USA.
- Urgaonkar, B., Shenoy, P., Chandra, A., Goyal, P., and Wood, T. (2008). Agile dynamic provisioning of multi-tier internet applications. *ACM Trans. Auton. Adapt. Syst.*, 3(1):1:1–1:39.
- Vijayakumar, S., Zhu, Q., and Agrawal, G. (2010). Dynamic resource provisioning for data streaming applications in a cloud environment. In *Proceedings of the 2010 IEEE Second International Conference on Cloud Computing Technology and Science*, pages 441–448, Washington, DC, USA.